ASSIGNMENT 2 – CO2

Q5: DERIVE A GENERIC RANKING COMPONENT FROM A PRINCIPLE

Inter-College Sports Leaderboard

This report intentionally excludes student-identifying details.

1. Problem and Ranking Principle

The sports meet ranks teams by three ordered criteria: points in descending order, goal difference in descending order, and team name in ascending alphabetical order. The supplied principle requires a consistent comparison: if A ranks above B and B ranks above C, A must rank above C; equal items must compare as equal.

2. Design Decision: Comparator or Comparable

Separate Comparator objects are preferred. A Team is a data object and may be ranked differently in different contexts (for example, points-first, goal-difference-first, or alphabetical). Implementing Comparable would attach one natural ordering to Team. A Comparator keeps the ranking policy outside the class and allows multiple reusable policies without changing Team.

The comparator chain is: points descending, then goal difference descending, then name ascending. `Comparator.comparingInt(...).reversed()` expresses each descending numeric criterion, `thenComparing(...)` applies the next criterion only when the preceding comparison is equal, and the final name comparison is alphabetical.

3. Generic Leaderboard Design

Leaderboard stores values of any reference type T and receives a `Comparator`. The bounded wildcard `? super T` allows a comparator that compares T or a supertype of T. The `addAll(List)` method accepts a list containing T or a subtype of T. The ranking method copies the internal list, sorts the copy with Collections.sort, and returns the copy. Thus, asking for a ranking does not reorder the original insertion list.

Pseudocode

```
Leaderboard<T> holds List<T> items and Comparator<? super T> order
add(T item): append item to items
addAll(List<? extends T> values): append values
ranking(): copy = new ArrayList(items); sort copy using order; return copy
```

4. Test Design and Correctness

Test 1 – Assignment dataset. Panthers(9,4), Eagles(9,6), Cobras(6,1), Bisons(9,4). Expected ranking: Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1). Eagles is first because it has the greatest goal difference among the teams with 9 points. Bisons is above Panthers because their points and goal difference are equal, and Bisons comes alphabetically first.

Test 2 – Designed multi-level ties. The program includes teams tied on points, teams tied on both points and goal difference, and teams with lower points. This verifies that each tie-breaker is applied only when the earlier criterion is equal. A triple-loop check tests transitivity over every triple in this test dataset.

Test 3 – Faulty comparator. A deliberately incorrect comparator compares points only. It fails to enforce goal difference and alphabetical order when points tie. The result may retain an incidental order for tied teams, but it does not implement the required ranking policy.

Overflow test. Subtracting one points value from another can overflow for extreme integers and reverse the intended result. Use `Integer.compare` or `Comparator.comparingInt` instead of subtraction.

5. Why the Comparator Chain Is Consistent

Each individual criterion uses a valid total ordering: integer order for points, integer order for goal difference, and lexicographic order for names. The chain is lexicographic: the first non-zero criterion determines the result; if it is zero, the next criterion is considered. Lexicographic composition of consistent total orders is transitive. If all three comparisons are zero, the teams have equal points, equal goal difference and equal name, so the comparator returns zero. The comparator avoids subtraction and therefore avoids integer-overflow errors.

6. Alternative Concept and Complexity Analysis

Approach	How it works	Time to rank n items	Extra space
Comparator chain + one sort	Sort once using all three criteria	O(n log n)	O(n) copy; sorting workspace implementation-dep
Comparable<Team>	Team defines one natural order; sort using natural ordering	O(n log n)	O(n) copy
Three stable sorts	Sort name A–Z, then goal difference descending, then points descending	3 × O(n log n) = O(n log n)	O(n) copy; sort workspace implementation-depend

Comparable alternative: Team can implement Comparable<Team> and define compareTo using the same chain. This is appropriate only if one natural ranking is universally correct for Team. It is less flexible when different competitions need different ranking policies.

Three stable sorts: First sort by name ascending, then by goal difference descending, then by points descending. Each later stable sort preserves the previous order among equal keys, so the final priority is points, goal difference, name. This is correct only when the sorts used are stable; it performs three sorting passes rather than one comparator-chain sort.

7. Experimental Evidence

The Java program measures sorting a copy of generated Team records at 1,000, 10,000 and 50,000 items. It performs one warm-up and five measured repetitions per size. Raw nanoseconds and milliseconds are written to task5_results.csv. The table reports the average measured milliseconds from the prepared execution. These are environment-specific results, not a promise of identical timings on another computer.

1,000 teams – average sort time: 1.1197 ms

10,000 teams – average sort time: 4.8092 ms

50,000 teams – average sort time: 25.0470 ms

The measured operation is sorting only; dataset generation and copying occur outside the timed interval. Since comparison sorting is O(n log n), larger datasets are expected to require more time. Actual values are affected by JVM warm-up, garbage collection, CPU scheduling and hardware.

8. Execution Procedure

Open a terminal in the Task05 folder. Compile with `javac LeaderboardDemo.java` and run with `java LeaderboardDemo`. No keyboard input is required. Verify the supplied ranking, transitivity check, faulty comparator demonstration and overflow demonstration. The program generates task5_results.csv. Capture a screenshot of your own local execution for submission.

9. Conclusion

A generic Leaderboard with an externally supplied Comparator is reusable and separates ranking policy from the Team data class. A lexicographic comparator chain expresses the three required criteria and is transitive. Returning a sorted copy preserves the stored insertion order. The deliberately faulty comparator demonstrates why every specified tie-breaker matters. The comparator-chain design is preferred when ranking rules may vary; Comparable is reasonable only when one natural order is appropriate.